1. Bubble short

- Pseudocode

```
PROCEDURE BubbleSort(arr, n)
  FOR i FROM 0 TO n-2 DO
    swapped = FALSE
    FOR j FROM 0 TO n-2-i DO
      IF arr[j] > arr[j+1] THEN
        SWAP(arr[j], arr[j+1])
        swapped = TRUE
      END IF
    END FOR
    IF swapped = FALSE THEN
      BREAK  // Array sudah terurut
    END IF
  END FOR
END PROCEDURE
```

- Code python

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

data = [64, 34, 25, 12, 22, 11, 90]

print("Sebelum:", data)
print("Sesudah:", bubble_sort(data))
```

```
Sebelum: [64, 34, 25, 12, 22, 11, 90]
Sesudah: [11, 12, 22, 25, 34, 64, 90]
```

- Analisis kompleksitas

Kasus	**Kompleksitas**
Best	O(n)
Average	$O(n^2)$
Worst	$O(n^2)$
Space	O(1)

2. Selection short

- Pseudocode

```
PROCEDURE SelectionSort(arr, n)
  FOR i FROM 0 TO n-2 DO
    min_idx = i
    FOR j FROM i+1 TO n-1 DO
      IF arr[j] < arr[min_idx] THEN
        min_idx = j
      END IF
    END FOR
    IF min_idx != i THEN
      SWAP(arr[i], arr[min_idx])
    END IF
  END FOR
END PROCEDURE
```

- Code python

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

data = [64, 25, 12, 22, 11]
print("Sebelum:", data)
print("Sesudah:", selection_sort(data))
```

```
Sebelum: [64, 25, 12, 22, 11]
Sesudah: [11, 12, 22, 25, 64]
```

- Analisis kompleksitas

Kasus	**Kompleksitas**
Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	O(1)

3. Insertion short

- Pseudocode

```
PROCEDURE InsertionSort(arr, n)
  FOR i FROM 1 TO n-1 DO
    key = arr[i]
    j = i - 1
    WHILE j >= 0 AND arr[j] > key DO
      arr[j+1] = arr[j]
      j = j - 1
    END WHILE
    arr[j+1] = key
  END FOR
END PROCEDURE
```

- Code python

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

data = [12, 11, 13, 5, 6]
print("Sebelum:", data)
print("Sesudah:", insertion_sort(data))
```

```
Sebelum: [12, 11, 13, 5, 6]
Sesudah: [5, 6, 11, 12, 13]
```

- Analisis kompleksitas

Kasus	**Kompleksitas**
Best	O(n)
Average	$O(n^2)$
Worst	$O(n^2)$
Space	O(1)

4. Merge short

- Pseudocode

```
PROCEDURE MergeSort(arr, left, right)
  IF left < right THEN
    mid = (left + right) / 2
    MergeSort(arr, left, mid)
    MergeSort(arr, mid+1, right)
    Merge(arr, left, mid, right)
  END IF
END PROCEDURE

PROCEDURE Merge(arr, left, mid, right)
  L = arr[left..mid]
  R = arr[mid+1..right]
  i = 0
  j = 0
  k = left
  WHILE i < len(L) AND j < len(R) DO
    IF L[i] <= R[j] THEN
      arr[k] = L[i]
      i = i + 1
    ELSE
      arr[k] = R[j]
      j = j + 1
    END IF
    k = k + 1
  END WHILE
  WHILE i < len(L) DO
    arr[k] = L[i]
    i = i + 1
    k = k + 1
  END WHILE
  WHILE j < len(R) DO
    arr[k] = R[j]
    j = j + 1
    k = k + 1
  END WHILE
END PROCEDURE
```

- Code python

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

data = [38, 27, 43, 3, 9, 82, 10]
print("Sebelum:", data)
print("Sesudah:", merge_sort(data))
```

```
Sebelum: [38, 27, 43, 3, 9, 82, 10]
Sesudah: [3, 9, 10, 27, 38, 43, 82]
```

- Analisis kompleksitas

Kasus	**Kompleksitas**
Best	O(n log n)
Average	O(n log n)
Worst	O(n log n)
Space	O(n)